SRI LANKA INSTITUTE OF INFORMATION TECHNOLOGY

Faculty of Computing — BSc (Hons) in Information Technology

SE4041 — Mobile Application Design and Development



FreshKeep

Eat it before it's too late — an on-device AI food freshness tracker

Final Report

Item	Details
Student name	Abisheak S
Student ID	IT21244384
Assignment	Assignment 02 — Swift (Part A: iOS · Part B: visionOS)
GitHub repository	github.com/IT21244384/SE4041-FreshKeep
Submission date	14 October 2026

Table of Contents

1. Executive Summary 3
2. Introduction 3
2.1 Problem 3
2.2 Purpose and target audience 3
3. Requirements and How They Were Met 3
4. Design Decisions 5
4.1 Architecture 5
4.2 UX decisions 6
5. User Interface 6
6. Implementation 7
6.1 Food recognition pipeline 7
6.2 Reading best-before dates 7
6.3 Persistence and reminders 8
6.4 Custom components 8
6.5 Adaptive layout 8
7. Testing 8
7.1 Strategy 8
7.2 Automated test results 9
7.3 Performance results 9
7.4 User acceptance testing 9
8. Performance Optimisation 10
9. Development Challenges 10
10. Deployment and Demonstration 11
11. Part B — FreshKeep Spatial (visionOS) 11
12. Use of AI Tools 12
13. Reflection 12
14. Conclusion 13
15. References 13
Appendix A — User Guide 14
Appendix B — Technical Documentation 17

1. Executive Summary

FreshKeep is a SwiftUI iOS app that reduces household food waste. Users photograph their groceries; Apple's Vision framework (Core ML) recognises the food on the device and FreshKeep estimates how long it will last based on where it is stored. It can also read printed best-before dates, sends reminders before food expires, suggests recipes that use up what is about to spoil, and reports the food, money and emissions the user has saved. A visionOS prototype, FreshKeep Spatial, extends the idea to a 3D virtual fridge and an immersive expiry timeline (Part B).

Outcome	Result
Screens	9 iPhone screens + iPad sidebar layout, Dark Mode and Dynamic Type variants
Emerging technology	Vision `VNClassifyImageRequest` (Core ML) + `VNRecognizeTextRequest` OCR, fully on device
Persistence	SwiftData model with external photo storage; settings in @AppStorage
Automated tests	33 core unit tests, 19 app unit tests, 10 UI tests — 62 of 62 passing in CI
Code coverage (app target)	83.2% (4,152 of 4,990 lines)
visionOS prototype	Builds and launches in the Apple Vision Pro simulator in CI
Repository	github.com/IT21244384/SE4041-FreshKeep (feature branches, pull requests, CI on every push)

2. Introduction

2.1 Problem

Households are the largest single source of food waste; UNEP's Food Waste Index Report 2024 estimates about 79 kg per person per year. Most of it is wasted because people forget what they have and when it expires. Existing pantry apps depend on manual entry, which users abandon, and loose produce has no printed date to copy.

2.2 Purpose and target audience

FreshKeep's purpose is to make tracking effortless and to nudge the user at the right time. It targets people who shop for themselves or a family — busy parents, students on a budget and older users who prefer large text — as described by the three personas in the proposal. Local context was considered: prices are in LKR, dates on packaging are read day-first, and the catalog includes local staples (coconut, brinjal, green chilli, cooked rice, curry).

3. Requirements and How They Were Met

Assignment requirement	Implementation	Where
Purpose & target audience	Food-waste problem, personas, local adaptation	Proposal §2–4

Assignment requirement	Implementation	Where
Advanced UI elements	Keyframe confetti, phase-animated onboarding, animated ring and scan line, 3D scroll transition, swipe / long-press / pinch gestures, adaptive layouts	`Views/`, `DesignSystem/Effects.swift`
SwiftUI components & customisation	FreshnessRing, `.freshCard()` modifier, button styles, ViewfinderShape, FilterChip, ShakeEffect; brand theme	`DesignSystem/`
≥3 navigable screens	Pantry, Item detail, Add/Edit, Scan, Insights, Recipe, Settings, Onboarding	`Views/`
Navigation & architecture	NavigationStack per tab with typed destinations, NavigationSplitView on iPad, sheets; MVVM + services + shared package	`RootView.swift`
Data persistence	SwiftData @Model, predicates and sort descriptors, external storage	`Shared/PantryItem.swift`
Emerging technology	Vision / Core ML food recognition and OCR	`Services/FoodClassifier.swift`, `LabelReader.swift`
Visual design & branding	Custom icon, colour tokens, Figma design system	Figma file, `Resources/Assets.xcassets`
Accessibility	Dark Mode assets, Dynamic Type, VoiceOver labels, Reduce Motion	UI Documentation §6–7
Testing & optimisation	Unit, UI and performance tests, UAT plan; ImageIO downsampling, caching, background Vision	§7–8
Documentation	User guide, technical documentation, this report, UI documentation, AI log	Appendices, `docs/`
Deployment & presentation	CI builds, simulator recording, README build steps, demo script	§10, README

4. Design Decisions

4.1 Architecture

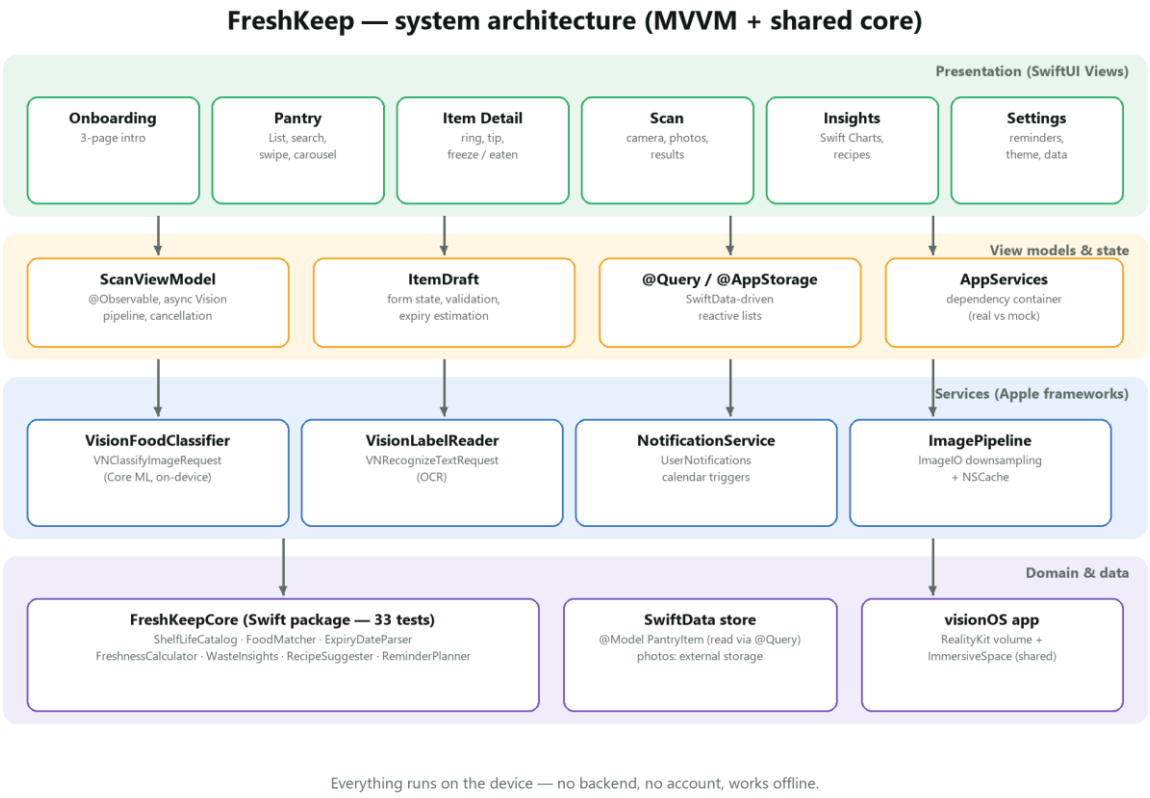


Figure 1 — FreshKeep architecture

MVVM with protocol-based services. Views stay declarative; `ScanViewModel` owns the asynchronous Vision pipeline; services hide Apple frameworks behind protocols so tests and previews can inject mocks. A tiny dependency container (`AppServices`) is passed through the SwiftUI environment.

A shared Swift package (FreshKeepCore). All business logic that does not need UIKit — the food catalog, label matcher, date parser, freshness maths, insights, recipe ranking and reminder timing — lives in a package used by both the iOS and visionOS apps. This made the logic fast to test (it runs with `swift test` on any machine, including my Windows laptop) and kept views thin.

SwiftData over Core Data or Realm. SwiftData gives Core Data's proven SQLite store with a Swift-native model, `@Query` for reactive lists and `#Predicate` filters, with no third-party dependency. Enums are stored as raw strings because predicates cannot use computed properties. Eaten and wasted items are kept rather than deleted so the Insights screen has history.

On-device ML instead of a cloud API. Vision's built-in classifier runs on the Neural Engine, works offline, costs nothing per request and keeps photos of the user's kitchen private. The trade-off is a generic label set, which FreshKeep compensates for with the synonym-aware `FoodMatcher`, a category fallback and always-editable results.

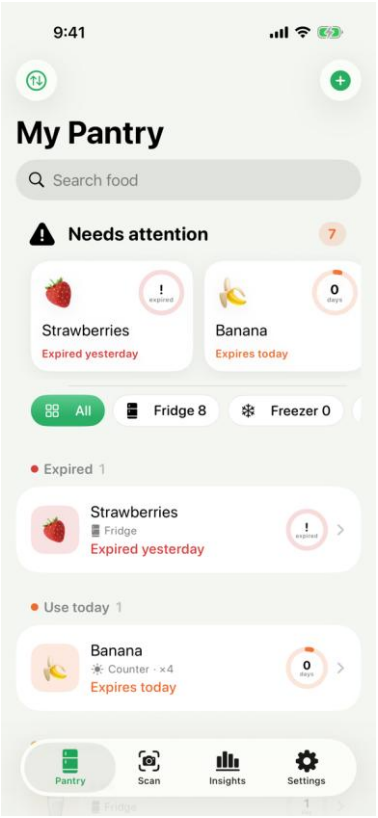
XcodeGen. The Xcode project is generated from `project.yml`. This avoids merge conflicts in `.pbxproj` files and, crucially, let the project be created and built from a clean checkout on CI without ever opening Xcode.

4.2 UX decisions

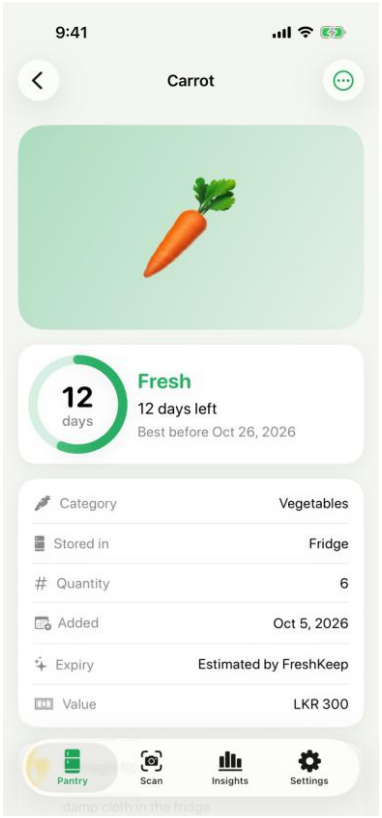
- **Urgency first** — the home screen is sorted and grouped by days left, with a "Needs attention" carousel, because the core job is deciding what to eat first.
- **Estimates over empty fields** — every item gets a sensible expiry immediately; the user only corrects it if needed. The form explains where the date came from.
- **Never colour alone** — every freshness colour is paired with text for colour-blind users and VoiceOver.
- **Positive reinforcement** — rescuing food triggers confetti, a haptic and a toast; Insights shows money saved rather than only waste.
- **Permission in context** — notification permission is requested at the end of onboarding, after explaining why.

5. User Interface

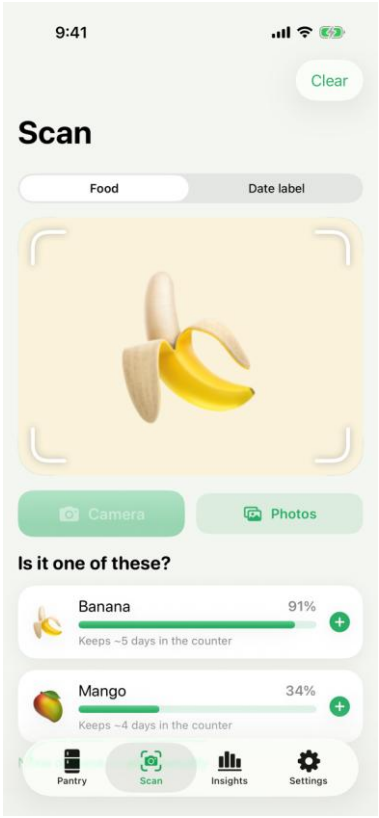
The UI was designed in Figma (design system, components and screens) and implemented in SwiftUI; the separate UI Documentation pairs each Figma frame with the implemented screen. The screenshots below were captured automatically by the UI tests on an iPhone 17 Pro simulator in CI.



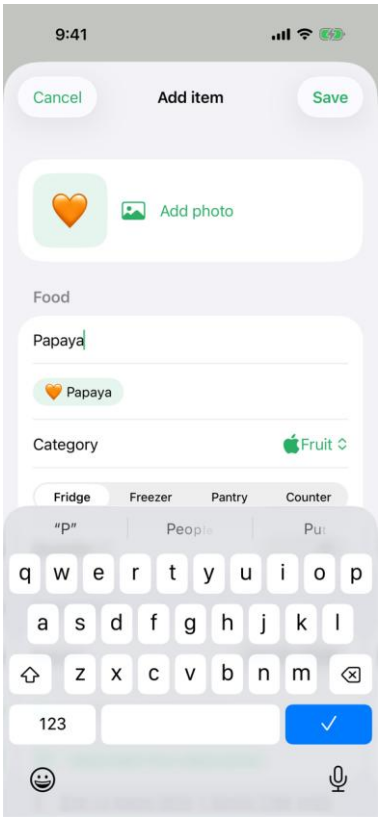
Pantry



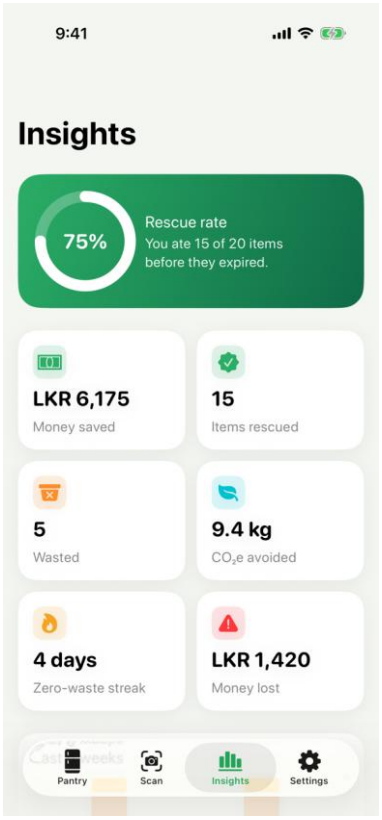
Item detail



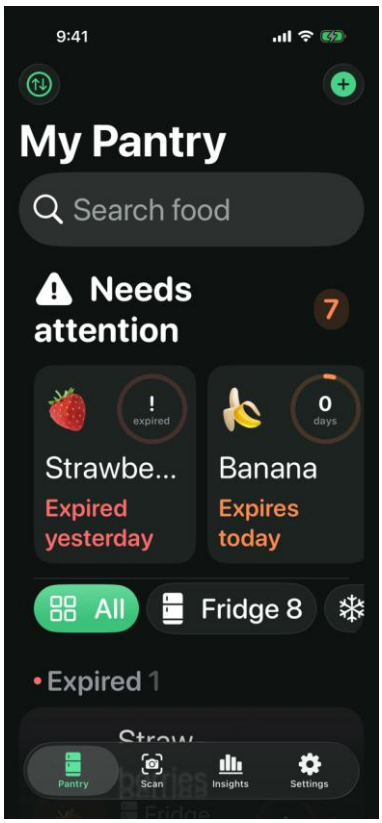
Scan results



Add item



Insights



Dark Mode + AX text

6. Implementation

6.1 Food recognition pipeline

Photos are downsampled to at most 1280 px before analysis. Vision runs in a detached task so the interface stays responsive; a new photo cancels the previous analysis.

```
let request = VNClassifyImageRequest()
#if targetEnvironment(simulator)
request.usesCPUOnly = true          // no Neural Engine in the Simulator
#endif
try VNImageRequestHandler(cgImage: cgImage, orientation: orientation).perform([request])
let observations = (request.results ?? [])
    .filter { $0.hasMinimumPrecision(0.1, forRecall: 0.8) || $0.confidence > 0.2 }
    .map { ClassifierObservation(identifier: $0.identifier, confidence: Double($0.confidence)) }
return FoodMatcher().matches(from: observations)   // foods only, synonyms merged
```

6.2 Reading best-before dates

`VNRecognizeTextRequest` (accurate mode, language correction off) returns text lines that `ExpiryDateParser` scans with ordered regular expressions: ISO dates, day–month-name–year, month-name–day–year, numeric day-first dates (swapping when the first number cannot be a month) and month/year dates, which mean the end of that month. When a label shows both a manufacture and an expiry date, the parser picks the first date that follows an expiry keyword (EXP, USE BY, BB…) and is not closer to an MFD keyword. Impossible dates such as 31/02/2026 are rejected — a case that the unit tests caught (see §9).

6.3 Persistence and reminders

`PantryItem` is a SwiftData `@Model`. Lists use `@Query(filter: #Predicate { $0.statusRaw == "active" }, sort: \.expiresAt)` so filtering and sorting happen in SQLite. When an item is saved, `ReminderPlanner` computes a "use soon" reminder N days before expiry and one on the morning of expiry; `NotificationService` schedules them with calendar triggers and cancels them when the item is eaten, wasted or deleted.

6.4 Custom components

```
struct FreshnessRing: View {
    let freshness: Freshness
    @ScaledMetric private var scale: CGFloat = 1        // grows with Dynamic Type
    @State private var animatedFraction: Double = 0
    var body: some View {
        ZStack {
            Circle().stroke(color.opacity(0.18), lineWidth: lineWidth)
            Circle().trim(from: 0, to: animatedFraction)
                .stroke(color.gradient, style: StrokeStyle(lineWidth: lineWidth, lineCap: .round))
                .rotationEffect(.degrees(-90))
            // days label …
        }
        .onAppear { withAnimation(.easeOut(duration: 0.9)) { animatedFraction =
freshness.remainingFraction } }
        .accessibilityValue("\(freshness.level.title), \(freshness.summary)")
    }
}
```

6.5 Adaptive layout

`RootView` checks the horizontal size class: compact width shows a `TabView`; regular width shows a `NavigationSplitView` sidebar with the same sections. The item detail uses `ViewThatFits` to place cards side by side when there is room, and the scan screen switches between vertical and horizontal stacks with `AnyLayout` in landscape.

7. Testing

7.1 Strategy

Level	Tool	Scope
Unit — core	XCTest via `swift test` (Windows + macOS CI)	Parser, matcher, catalog, freshness, insights, recipes, reminders
Unit — app	XCTest (iOS Simulator)	SwiftData queries, resolve/freeze, drafts, scan view model with mocks, image pipeline
UI	XCUITest	End-to-end flows with an in-memory store and mock classifier for repeatability
Performance	XCTest `measure`, launch metric	Catalog lookups, 500-item fetch, app launch
User acceptance	Task-based sessions	Five participants, scripted tasks, satisfaction questions

7.2 Automated test results

Suite	Tests	Passed	Notes
FreshKeepCoreTests (package)	33	33	Run on Windows (Swift 6.4.1) and macOS CI
FreshKeepTests (app unit)	19	19	SwiftData, drafts, scan view model, image pipeline
FreshKeepUITests	10	10	Flows, screenshots, dark mode + AX text, launch

Code coverage for the FreshKeep app target (unit + UI tests): **83.2% (4,152 of 4,990 lines)**. All automated tests run on every push in GitHub Actions on an iPhone 17 Pro simulator (iOS 26.2) with Xcode 26.3.

Selected unit test cases

Test	What it proves
testPrefersExpiryOverManufactureDate	"MFD 01/10/2026 EXP 01/04/2027" → 1 April 2027
testRejectsImpossibleDates	31/02/2026 is not a date (regression test for a bug found during development)
testMonthAndYearOnlyMeansEndOfMonth	"EXP 02/2028" → 29 February 2028 (leap year)
testMergesAliasesAndKeepsHighestConfidence	"granny_smith" 62% and "apple" 30% → one Apple match at 62%
testPrefersRecipesUsingUrgentItems	A banana expiring today ranks Banana Smoothie first
testFreezeExtendsExpiryAndMovesToFreezer	Freezing chicken extends expiry and cannot be applied twice
testFoodScanProducesMatchesAndDownsamples	Scan state machine reaches results; working image ≤ 1280 px

7.3 Performance results

Measurement	Result	How
Fetch 500 items (predicate + sort)	0.013 s	XCTest measure, iPhone 17 Pro simulator
App launch (cold, UI-test build)	3.56 s	XCTApplicationLaunchMetric on the CI simulator (shared virtual Mac, slower than a device)
2,000 catalog lookups	0.019 s (≈10 µs each)	XCTest measure, Windows laptop (debug build)
Stored photo size	≤ 1024 px JPEG	ImagePipeline test

7.4 User acceptance testing

UAT is run with five participants who match the personas, using the app on an iPhone. Each participant completes the tasks below without help while thinking aloud; completion, time and errors are recorded, followed by three 1–5 rating questions.

#	Task	Success criterion
T1	Add a banana by taking a photo	Item saved with a suggested expiry in < 60 s
T2	Find which item must be eaten first	Opens the correct item from “Needs attention”
T3	Add milk using the date on the carton	Expiry read from the label and saved
T4	Mark an item as eaten	Uses swipe or the detail button
T5	Find how much money you have saved	Opens Insights and reads the value

Results table (to be completed after the sessions):

Participant	T1	T2	T3	T4	T5	Ease (1–5)	Comments
P1							
P2							
P3							
P4							
P5							

8. Performance Optimisation

- **Memory:** camera images (~12 MP, ~48 MB decoded) are never decoded at full size; ImageIO thumbnails are created directly from the compressed data. Stored photos are ≤ 1024 px JPEGs (~150 KB). Thumbnails are cached in an `NSCache` limited to 150 entries.
- **Storage:** photos use SwiftData external storage, so list fetches don’t load image blobs.
- **Responsiveness:** Vision runs in a detached task with user-initiated priority; stale analyses are cancelled.
- **Rendering:** lazy stacks/grids, value-type derived data, and animations limited to transforms and opacity.
- **Launch:** no network, no heavy work at launch; sample data is only inserted when requested.

9. Development Challenges

Challenge	What happened	Resolution
No Mac	Xcode only runs on macOS; my laptop runs Windows.	Wrote shared logic as a Swift package tested on Windows; generated the project with XcodeGen and built/tested it on GitHub Actions macOS runners on every push (including screen recordings and screenshots).
Date parser bug	“31/02/2026” was rejected, but a looser pattern then read “02/2026” as 28 Feb.	Matched text is now claimed even when the date is invalid. Regression test added.

Challenge	What happened	Resolution
Name clash on visionOS	`Scene` was ambiguous because RealityKit also defines `Scene`.	Qualified as `SwiftUI.Scene`.
Flaky UI tests	System permission alert and list-row lookups differ between OS versions.	Handled the alert through SpringBoard and looked up rows by identifier regardless of element type.
Simulator has no camera	The scan flow could not be exercised in CI.	Injected a mock classifier via a launch argument and added a “Try a sample photo” debug button.
Figma plan limits	The free plan allows 3 pages, 1 variable mode and a limited number of API calls.	Light/dark tokens as two variable groups; designs grouped into sections.

10. Deployment and Demonstration

The app builds from a clean checkout with `xcodegen generate` and Xcode 16 or later; README contains the steps. Every push produces build artifacts: UI-test screenshots, an MP4 recording of the UI tests running in the simulator, a code-coverage report and a visionOS screenshot and recording. For the live demonstration the app is run from Xcode on a Mac on an iPhone (camera scanning) and in the Apple Vision Pro simulator.

Demo script: onboarding → load sample pantry → swipe an item as eaten (confetti) → scan a banana → read a date from a milk carton → item detail and freeze → Insights and a recipe → Dark Mode and large text → visionOS fridge and timeline.

11. Part B — FreshKeep Spatial (visionOS)

FreshKeep Spatial reuses the SwiftData model and FreshKeepCore. It offers a glass main window with an ornament, a volumetric “virtual fridge” built from RealityKit entities generated in code (shelves per storage location, tokens coloured by freshness, a cooking pot) and a mixed immersive “expiry timeline”. Users look and pinch to see an item’s floating card, or pinch and drag it into the pot to mark it eaten and get a recipe idea. The concept, competitor analysis, revenue model and go-to-market plan are in the pitch deck.

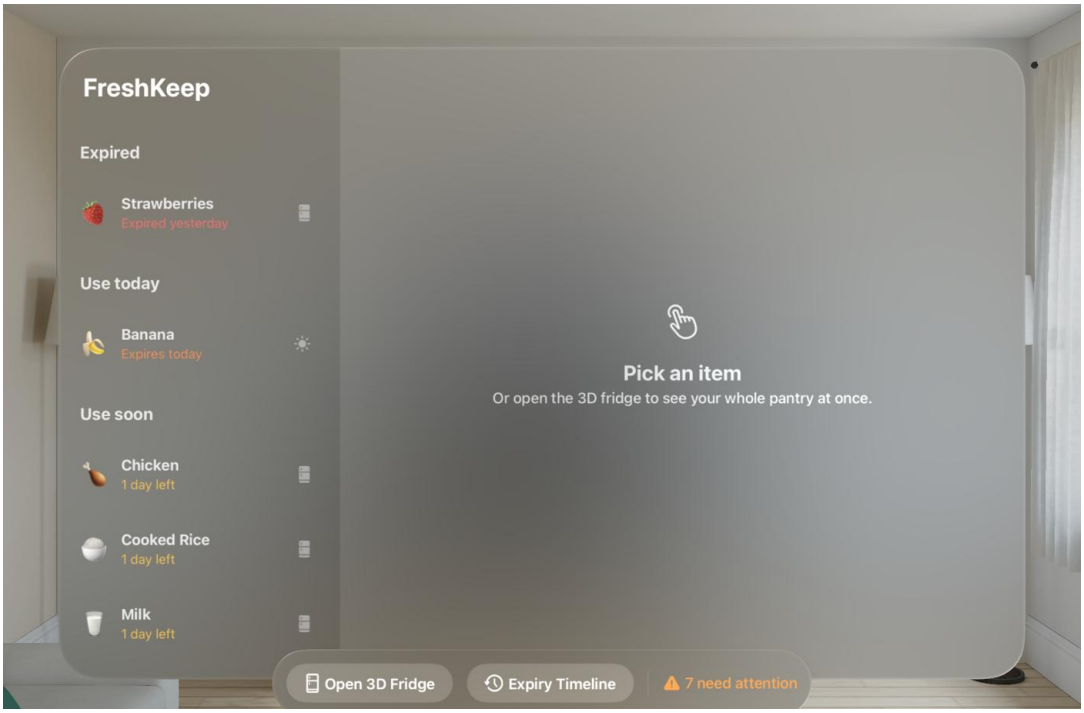


Figure — FreshKeep Spatial in the Apple Vision Pro simulator

12. Use of AI Tools

Claude (Anthropic) was used as a coding assistant, as permitted by the brief. It helped scaffold the project structure, CI scripts and asset catalogs, suggested edge cases for tests, and drafted parts of the documentation. I chose the problem, audience and features, reviewed and tested all generated code, and verified facts and shelf-life values. Prompts, outputs and how they were used are recorded in `docs/AI_USAGE_LOG.md` in the repository.

13. Reflection

The most valuable lesson was to separate logic from UI early. Putting the catalog, parser and statistics into a package let me test them in seconds without a simulator, and those tests caught a real bug before it reached the app. Designing the component library in Figma first also paid off: each component became one SwiftUI view or modifier, and the screens were quick to assemble.

Working without a Mac was the biggest constraint. Building through CI made each compile cycle slower (around ten minutes), which pushed me to write more carefully, rely on the compiler's strictness and test logic locally. If I repeated the project I would set up the CI pipeline on day one — the automatic screenshots and recordings were also very useful for documentation.

If I had more time I would train a Create ML model on local produce (jackfruit, gotukola, rambutan), add barcode lookup for packaged goods, iCloud sync for shared households and home-screen widgets for items that must be used today.

14. Conclusion

FreshKeep meets the assignment requirements with a polished, accessible SwiftUI app that integrates on-device machine learning in a way that is central to the product, persists data with SwiftData, follows a clean architecture, and is backed by automated tests and continuous integration. The visionOS prototype shows how the same core can power a spatial experience.

15. References

1. United Nations Environment Programme (2024). Food Waste Index Report 2024. Nairobi: UNEP.
2. Apple Inc. Vision framework. developer.apple.com/documentation/vision
3. Apple Inc. SwiftData. developer.apple.com/documentation/swiftdata
4. Apple Inc. Swift Charts. developer.apple.com/documentation/charts
5. Apple Inc. RealityKit and visionOS. developer.apple.com/visionos
6. Apple Inc. Human Interface Guidelines. developer.apple.com/design/human-interface-guidelines
7. USDA FSIS. FoodKeeper data. foodsafety.gov/keep-food-safe/foodkeeper-app
8. XcodeGen. github.com/yonaskolb/XcodeGen

Appendix A — User Guide

FreshKeep helps you eat your food before it goes off. Add groceries with your camera, get a reminder before they expire, and see how much food and money you've saved.

1. Getting started

1. Open *FreshKeep. Swipe through the three welcome pages (or tap Skip*).
2. Tap *Get started* and allow notifications so FreshKeep can remind you before food expires.
3. Your pantry is empty at first. Tap *Scan food or Add manually. Tip: Settings → Load sample pantry* fills the app with demo items so you can explore.

2. Adding food

Scan with the camera (fastest)

4. Open the *Scan tab and keep Food* selected.
5. Tap *Camera and take a photo of the item (or Photos* to pick one).
6. FreshKeep recognises the food on your phone and shows the best matches with a confidence bar.
7. Tap the right match. The form opens already filled in — name, category, storage and an estimated expiry date. Adjust anything and tap *Save*.
8. Not right? Tap *None of these — add manually*.

Read the date from a label

1. In *Scan, switch to Date label*.
2. Photograph the best-before / use-by date on the packaging.
3. FreshKeep shows the date it found (it ignores the manufacture date). Tap *Add item with this date, type the name and save. You can also tap Read date from label photo* inside the add form.

Add by hand

Tap **+** on the Pantry screen. Start typing a name — suggestions appear; picking one fills in the category, storage and expiry estimate. Changing **Stored in** (Fridge / Freezer / Pantry / Counter) updates the estimate automatically unless you set a date yourself.

3. Your pantry

- **Needs attention** at the top shows everything expiring today, soon, or already expired. Swipe through the cards and tap one to open it.
- Items are grouped by urgency: **Expired**, **Use today**, **Use soon**, **Fresh**. The ring shows how much time is left; the number is days remaining.
- Filter by **Fridge / Counter / Pantry / Freezer**, search by name, or change the sort order (⇅ button).
- **Swipe right** on an item → **Eaten**. **Swipe left** → **Wasted** or **Delete**.

- **Touch and hold** an item for a preview and quick actions (Eaten, Freeze it, Wasted, Delete).

4. Item details

Tap any item to see its freshness ring, when it expires, how the date was decided (estimated, read from label or entered by you), a storage tip and recipe ideas.

- **I ate it** — marks it as rescued (with a little celebration 🎉).
- **Freeze** — moves it to the freezer and extends its expiry date.
- **Wasted** — logs it so Insights can show what you tend to throw away.
- **⋯ → Edit / Delete** — change details or remove it without counting it.
- Pinch the photo to zoom.

5. Reminders

FreshKeep sends two reminders for each item: a "use soon" reminder (2 days before by default) and one on the morning it expires. Change the timing in **Settings → Notifications**, or turn reminders off.

6. Insights and recipes

The **Insights** tab shows your **rescue rate**, money saved (LKR), items rescued and wasted, CO_2e avoided, your zero-waste streak, a 6-week chart and which categories you waste most. **Use it up** lists recipes that use what's expiring next — tap one for ingredients and a step-by-step checklist.

7. Settings

Setting	What it does
Expiry reminders	Turn notifications on/off
Remind me N days before	Lead time for the "use soon" reminder
Reminder time	Time of day reminders arrive
"Use soon" when N days left	When items move into the *Use soon* group
Theme	System, Light or Dark
Load sample pantry	Adds demo items and history
Show welcome screens again	Replays onboarding
Erase all data	Permanently deletes all items and history

8. Accessibility

FreshKeep supports Dark Mode, Dynamic Type (all text grows with your system text size), VoiceOver (every control and the freshness ring are labelled) and Reduce Motion. Status is always shown as text as well as colour.

9. Privacy

Recognition runs entirely on your iPhone. Photos and pantry data never leave your device. There is no account and no internet connection is needed.

10. Troubleshooting

Problem	Try this
Food not recognised	Fill the frame with one item, use good light, or pick **None of these**
Date not found	Get closer to the date, avoid glare, keep the text horizontal
No reminders	Check Settings → Notifications in FreshKeep and in iOS Settings
Camera button disabled	The device has no camera (e.g. Simulator) — use **Photos**

11. FreshKeep Spatial (Apple Vision Pro prototype)

- The main window lists your pantry; pick an item to see details and recipes.
- **Open 3D Fridge** opens a volume with shelves. Look at an item and pinch to see its card; pinch and drag an item into the **Cook now** pot to mark it eaten and get a recipe idea.
- **Expiry Timeline** places food around you — the closer it floats, the sooner it expires.

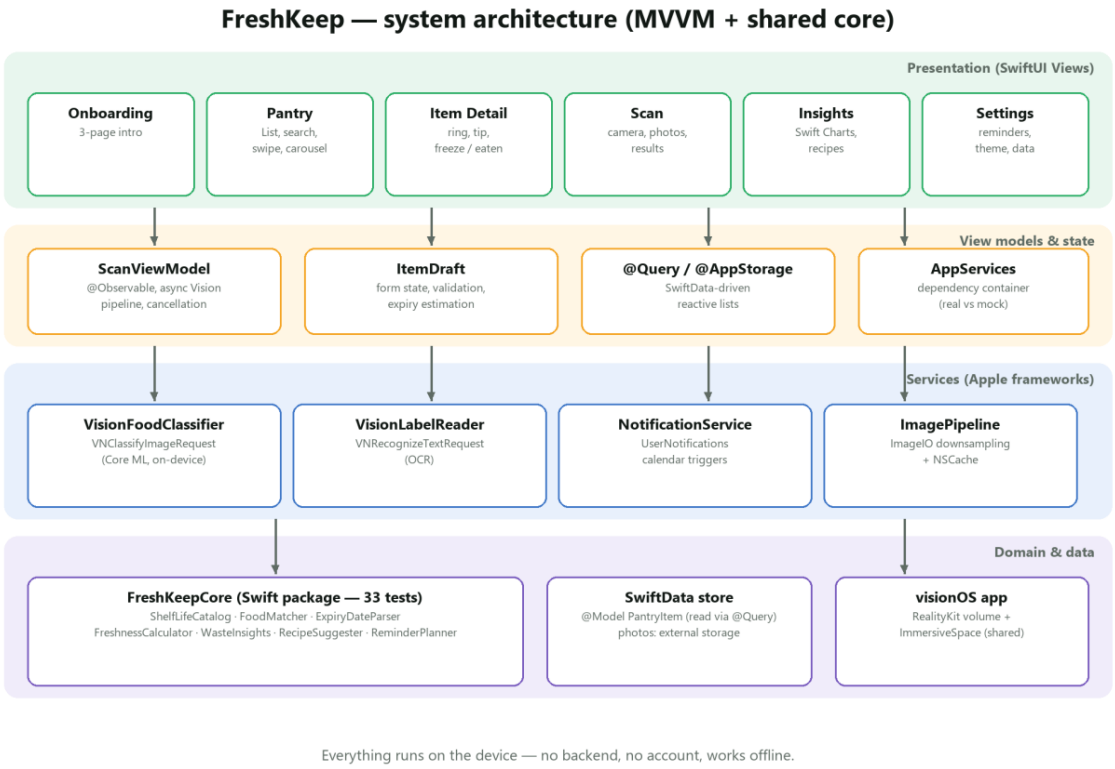
Appendix B — Technical Documentation

1. Overview

FreshKeep is a SwiftUI app for iOS 17+ (iPhone and iPad) plus a visionOS prototype. All logic runs on the device: there is no backend and no account.

Item	Value
Language	Swift (Swift 5 language mode, builds with Xcode 16 / 26)
UI	SwiftUI, Observation (`@Observable`), Swift Charts, PhotosUI
Persistence	SwiftData (`@Model PantryItem`), `@AppStorage` for settings
ML	Vision: `VNClassifyImageRequest` (Core ML image classifier), `VNRecognizeTextRequest` (OCR)
Notifications	UserNotifications with calendar triggers
visionOS	RealityKit `RealityView`, volumetric `WindowGroup`, `ImmersiveSpace`, ornaments
Project	XcodeGen (`project.yml`) → `FreshKeep.xcodeproj`
CI	GitHub Actions, macOS runners (`.github/workflows/ios.yml`)

2. Architecture



The app follows **MVVM with services**:

- **Views** (`FreshKeep/Views`) are small SwiftUI structs. Lists read SwiftData directly with `@Query`, which keeps them reactive without boilerplate.
- **View models / state** — `ScanViewModel` (`@Observable`, main-actor) owns the scan pipeline: downsampling, running Vision off the main thread, cancellation when a new photo arrives, and the `Phase` state machine (`idle → analyzing → foods | noFood | label | failed`). `ItemDraft` is a value-type form model so cancelling a sheet never touches the store.
- **Services** (`FreshKeep/Services`) wrap Apple frameworks behind protocols (`FoodClassifying`, `LabelReading`, `NotificationScheduling`). `AppServices` is injected through the SwiftUI environment and swaps in deterministic mocks for UI tests (`-mock-classifier`).
- **FreshKeepCore** (`Packages/FreshKeepCore`) is a Swift package with no UI dependencies. It is shared by the iOS and visionOS targets and tested on any platform with `swift test`.

2.1 Module responsibilities

Type	Responsibility
`ShelfLifeCatalog`	48 foods: aliases, shelf life per storage location, storage tip, typical LKR price. Normalises classifier labels (`granny_smith` → "granny smith"), handles plurals, prefix search.
`FoodMatcher`	Filters Vision labels to foods, merges synonyms, keeps highest confidence per food, falls back to a category ("fruit") when no specific food is found.
`ExpiryDateParser`	Regex-based parser for day-first numeric dates, ISO dates, month names, month/year-only dates (→ end of month). Chooses the date after an expiry keyword over MFD dates; rejects impossible dates.
`FreshnessCalculator`	Days left, level (expired / today / use soon / fresh) and remaining fraction for the ring.
`WasteInsights`	Totals, rescue rate, money saved/wasted, CO_2e estimate, weekly buckets, zero-waste streak.
`RecipeSuggester`	Scores recipes: `2 × Σ 1/(daysLeft+1)` for owned ingredients + coverage ratio.
`ReminderPlanner`	"Use soon" and "expires today" fire dates at a configurable hour, dropping past dates.

2.2 Data model

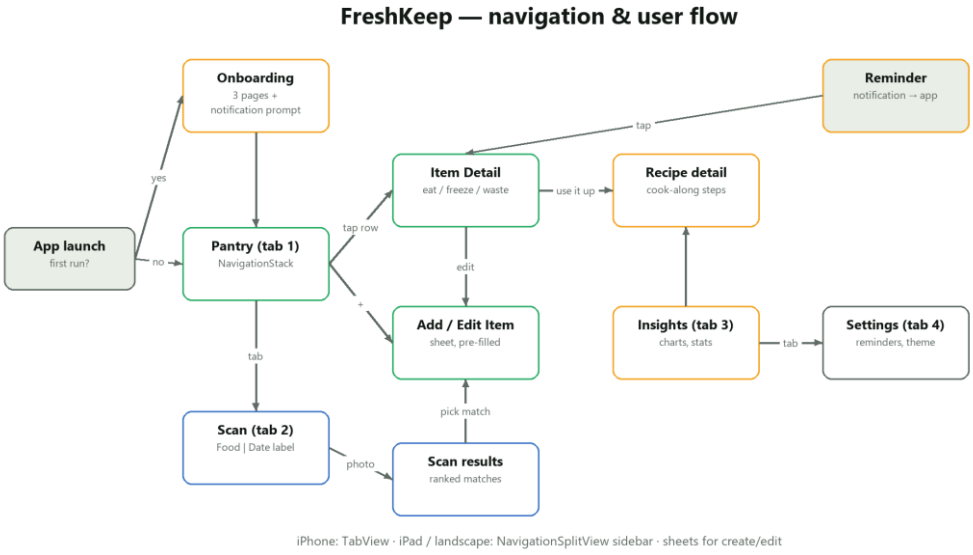
```
@Model final class PantryItem {
    @Attribute(.unique) var id: UUID
    var name: String
    var catalogID: String?          // link to ShelfLifeCatalog
    var categoryRaw: String         // FoodCategory
    var storageRaw: String          // StorageLocation
    var quantity: Int
    var addedAt: Date
```

```
    var expiresAt: Date
    var price: Double               // LKR per unit
    var notes: String
    @Attribute(.externalStorage) var photoData: Data?
    var statusRaw: String           // active | eaten | wasted
    var resolvedAt: Date?           // when eaten/wasted, drives charts
    var expirySourceRaw: String     // estimated | label | manual
}
```

Enums are stored as raw strings because `#Predicate` cannot use computed properties; typed accessors (`category`, `storage`, `status`) wrap them. Photos use external storage so list fetches stay small. Eaten/wasted items are kept (not deleted) to power the Insights screen.

2.3 Navigation

- iPhone (compact width): `TabView` with Pantry, Scan, Insights, Settings.
- iPad / large landscape (regular width): `NavigationSplitView` with the same sections in a sidebar.
- Each section owns a `NavigationStack`. Pantry uses value-based navigation (`NavigationLink(value:)` + `.navigationDestination(for: PantryItem.self)`), Insights navigates to `RecipeSuggestion` values. Create/edit flows are sheets; onboarding is a full-screen cover.



3. Emerging technology: Vision / Core ML

```
UIImage ──► downsample (≤1280 px) ──► VNImageRequestHandler
                │                          ├─ VNClassifyImageRequest ──► [label, confidence]
                │                          │        └─► FoodMatcher ──► ranked FoodMatch list
                │                          └─ VNRecognizeTextRequest ──► recognised lines
                │                                   └─► ExpiryDateParser ──► Date?
                └─ stored copy (≤1024 px JPEG, external storage)
```

- Requests run in `Task.detached(priority: .userInitiated)` so the UI stays at 60/120 fps.
- On the Simulator `usesCPUOnly` is set because there is no Neural Engine.
- Observations are filtered with `hasMinimumPrecision(0.1, forRecall: 0.8)` or confidence > 0.2.

- OCR uses `.accurate` recognition with language correction **off** (it "corrects" `12.11.26` into words otherwise).

4. UI implementation highlights

Requirement	Implementation
Custom component	`FreshnessRing` — trimmed circle, gradient stroke, animated on appear/change, `@ScaledMetric`, VoiceOver value.
Custom modifier	`FreshCardModifier` (`.freshCard()`), `ShakeEffect` (GeometryEffect for invalid input).
Custom styles	`PrimaryButtonStyle`, `SoftButtonStyle` (spring scale on press), `FilterChip`.
Custom shape	`ViewfinderShape` (corner brackets) + animated `ScanningOverlay`.
Animations	keyframe confetti (`keyframeAnimator`), `phaseAnimator` floating emoji, `symbolEffect`, `scrollTransition` 3D carousel, `contentTransition(.numericText())`, animated charts.
Gestures	swipe actions, long-press context menu with preview, `MagnifyGesture` pinch-zoom, visionOS spatial tap + drag.
Adaptive layout	`NavigationSplitView` vs `TabView`, `ViewThatFits` (detail cards), `AnyLayout` HStack/VStack (scan screen), adaptive `LazyVGrid`.
Feedback	`sensoryFeedback` haptics, toasts with VoiceOver announcements.
Accessibility	Dynamic Type everywhere (system text styles), `@ScaledMetric`, combined accessibility elements, labels/hints, reduce-motion checks, colour + text for status (never colour alone).
Dark Mode	Every colour is an asset with a dark variant; shadows removed in dark mode.

5. Performance and memory

- **Image downsampling with ImageIO** (`CGImageSourceCreateThumbnailAtIndex`) — a 12 MP photo is never decoded at full size. Stored photos are ≤1024 px JPEG (~150 KB instead of ~48 MB decoded).
- **Thumbnail cache** — `NSCache` (150 entries) keyed by item id and size.
- **External storage** for photos keeps SwiftData fetches fast; `@Query` uses a predicate and sort descriptor so filtering happens in SQLite.
- **Vision off the main actor**, with cancellation of stale scans.
- **Lazy containers** (`LazyHStack`, `LazyVGrid`) and value types for derived data.

- Measured in tests: catalog lookups (`measure`), 500-item fetch, app launch (`XCTApplicationLaunchMetric`).

6. Testing

Suite	Count	Covers
`FreshKeepCoreTests`	33	parser formats & edge cases, matcher, catalog, freshness, insights, recipes, reminders, performance
`FreshKeepTests`	19	SwiftData queries, resolve/freeze, record maths, sample data, item draft logic, scan view model (mock services), image pipeline, fetch performance
`FreshKeepUITests`	10	onboarding, pantry + detail, add item, validation, scan → add, label scan, mark eaten, insights/settings, dark mode + accessibility text size, launch performance

Run locally (`swift test`, ⌘U in Xcode) or let CI run them on every push. CI exports UI-test screenshots and a screen recording as build artifacts.

7. Build, run and CI

```
brew install xcodegen
xcodegen generate
open FreshKeep.xcodeproj         # schemes: FreshKeep (iOS), FreshKeepSpatial (visionOS)
```

Launch arguments (used by UI tests and demos): `-ui-testing` (in-memory store), `-skip-onboarding`, `-seed-sample-data`, `-mock-classifier`, `-appearance dark`.

CI jobs: `core` (swift test), `ios` (xcodegen → xcodebuild test on the newest iPhone simulator, screen recording, screenshots, coverage), `visionos` (build for visionOS simulator, launch, screenshot, recording).

8. Privacy and security

No network calls, analytics or accounts. Camera and photo access are requested with clear usage descriptions. Data lives in the app sandbox and can be erased from Settings.

9. Known limitations / future work

- Apple's generic classifier recognises common produce well but not every packaged product; a custom Create ML model trained on local foods (e.g. jackfruit, gotukola) would improve accuracy.
- Barcode lookup (VisionKit `DataScannerViewController`) for packaged goods.
- iCloud sync (SwiftData + CloudKit) and a shared household pantry.
- Widgets / Live Activities for "use today" items.